Compiler Design

CO-1 Assessment Tool-2

S.A. Mohammed Fareeth
192511288
CSA1401

1) a) Error identification

Line no.	Error	Type
5	i<=n should be i<n	logical / Runtime error
15	sumArray (arr); missing second argument	Semantic error
18	Missing ; after printf	Syntax error
22	a/b where b=0	Runtime error
24	int *p; pointer not initialized	Runtime
25	*p = 20; dereferencing uninitialized pointer	Runtime

b) Error Classification

* Lexical Error: None
* Syntax error: Incorrect if syntax, missing semicolon (Line 18)
* logical error: Wrong loop condition i<= n (Line 5)
* Runtime error: Division by zero (Line 22)
  uninitialized pointer (Line 24-25)
  array out of bounds (Line 5)

c) Explain compiler behavior

* Compile-time errors: Missing semicolon and incorrect function arguments

* Run-time errors: Division by zero, invalid pointer access, array out of bounds

The compiler stops code generation until syntax and semantic errors are fixed

d) Corrected Code:

```
#include <stdio.h>
int sumArray(int arr[], int n){
    int i, sum=0;
    for(i=0; i<n; i++){
        sum += arr[i];
    }
    return sum;
}
int main(){
    int arr[5] = {1,2,3,4,5};
    int total = sumArray(arr, 5);
    if(total == 15) printf("Correct Sum\n");
    int a=10, b=2;
    int c = a/b;
    int value = 20;
    int *p = &value;
    *p = 20;
    return 0;
}
```

e) Impact of the errors

1) Array out of bounds: Incorrect output or crash
2) Wrong function arguments: Compilation fails
3) Missing semicolon: Compilation error
4) Divison by zero: Program crashes
5) Uninitialized pointer: Segmentation fault or undefined behavior

Compiler improvements

* Display exact line numbers in error messages
* Suggests possible fixes (e.g. Did you mean 'i' n?)
* Detect possible runtime errors using static analysis
* Warn about division by zero and initialized pointer before execution

2) a) Error identification

Line	Invalid Token	Reason
2	1 num	identifier cannot start with digit
3)	@	illegal symbol in identifier
4)	'abc'	character literal contains more than one character
5	x0 $ 20	$ is an illegal symbol
6	num #1	# is not allowed in identifiers
7	% count	Identifiers cannot start with %
11	09	Invalid octal constant (9 not allowed)
12	0xz12	z is invalid in a hexadecimal
13	12.3.4	Invalid floating point constant
17	invalid-id	"-" is not allowed in identifier
20	x0 @5	@ is illegal operator/symbol
22	'xy'	character literal has more than one character
23	3value	Identifier cannot start with a digit

b) Error Classification

1) 1num, 3value - identifier starts with digit
2) rate@, num#1, %count, invalid-id - Invalid characters in iden
3) 'abc', 'xy' - Character literal must contain only one character
4) 09, 0x212, 12.3.14 - Invalid constant formation
5) 50$20, 10@5 - illegal symbols / operators

c) Token correction

1num → num1
rate @ → rate
'abc' → 'a'
50$20 → 5020
num#1 → num1
%count → count

09 → 9
0x212 → 0xA12
12.3.4 → 12.34
invalid-id → invalid_id
10@5 → 10+5
'xy' → 'x'
3value → value3

d) Compiler Behaviour

1) The lexical analyzer (scanner) reads the source code and converts it into valid tokens
2) Invalid tokens produce lexical error messages
3) Most compilers continue scanning using error recovery to report multiple lexical errors instead of stopping at first one

e) Advanced Thinking

(i) Simple DFA rule for identifiers:
* First character must be A-Z, a-z, or _
* Remaining characters can be letters, digits or _
* If first character is a digit, report a lexical error

OR

Better error messages:
* Show the line number
* Highlight invalid token
* Explain the reason
* Suggest a correction

Identify the Errors

Line	Error	Type
5	Missing ;	Syntax Error
9	# Value invalid Identifier	Lexical Error
13	divide (10) missing argument	Semantic Error
16	Missing ; after printf()	Syntax Error
20	Division by zero	Runtime Error
22	arr[4] out of bounds	Runtime Error
25	Uninitialized pointer	Runtime
28	+0 unnecessary	Optimization
30	infinite loop	logical

b) Compiler Phase

* Lexical : #value
* Syntax : Missing ;
* Semantic : Wrong function arguments
* Optimization : Remove +0
* Runtime : Divide by zero, pointer error, array overflow

c) Compile vs Run Time

* Compile time: Invalid identities, missing ;, wrong arguments.
* Runtime : Division by zero, invalid pointer, array out of bounds, infinite loop

d) Corrected Program

```
#include <stdio.h>
int divide (int a, int b) {
    return a/b;
}
int main() {
    int value = 20;
    float num = 15.5f
    int result;
    result = divide (10, 2);
    if (result == 5) printf ("Result is 5 \n");
    int x = 10, y = 2;
    int z = x/y;
    int arr[4] = {1, 2, 3, 4}
    printf ("%d\n", arr[3]);
    int data = 25;
    int *ptr = &data; *ptr = 25;
    int total = x+y * result;
    while (x < 20) {
        printf ("%d\n", x);
        x++;
    }
    return 0;
}
```

e) Impact:

* Invalid pointer → Crash
* Array overflow → Undefined behaviour
* Division by zero → Runtime error

f) Improvements

* Better error messages with line number
* Detect runtime issues using static analysis
* Optimized code by removing unnecessary expressions